

Zrównoleglenie algorytmu HGS dla problemu CVRP

Analiza wpływu GPU, modelu master-slave i modelu wysp na jakość rozwiązań

Hubert Blonowski, Jan Zaborski

14 czerwca 2026

Plan prezentacji

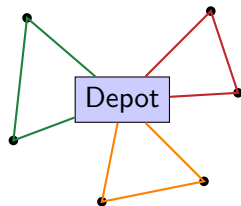
- 1 Problem CVRP
- 2 Algorytm HGS
- 3 Wprowadzone modyfikacje
- 4 Wyniki eksperymentów
- 5 Spojrzenie biznesowe
- 6 Wnioski

Czym jest CVRP?

Capacitated Vehicle Routing Problem — kanoniczny problem optymalizacji tras.

Dane:

- jeden magazyn (depot) i n klientów,
- każdy klient ma zapotrzebowanie q_i ,
- flota pojazdów o jednakowej pojemności Q ,
- macierz odległości (kosztów) między punktami.



Cel: znaleźć zbiór tras (każda zaczyna i kończy się w depot) o **minimalnej łącznej długości**, tak aby:

- każdy klient był odwiedzony dokładnie raz,
- suma zapotrzebowań na trasie $\leq Q$.

Dlaczego to trudny problem?

- CVRP jest **NP-trudny** — uogólnia problem komiwojażera (TSP).
- Przestrzeń rozwiązań rośnie wykładniczo wraz z liczbą klientów.
- Dla instancji z tysiącami klientów dokładne metody są bezużyteczne — stosuje się **metaheurystyki**.
- Praktyczne znaczenie: logistyka, kurierzy, dystrybucja, zbiórka odpadów — drobna poprawa % tras = realne oszczędności paliwa i czasu.

State of the art

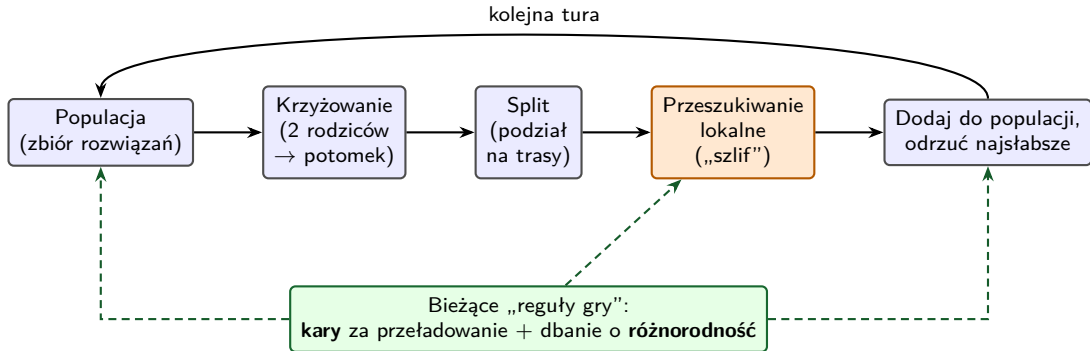
HGS (Hybrid Genetic Search) Vidala i in. jest jednym z najlepszych znanych algorytmów dla CVRP — bardzo dobre rozwiązania w krótkim czasie.

Główna idea: „hodujemy” rozwiązania jak w ewolucji

Nie da się sprawdzić wszystkich możliwych zestawów tras — jest ich astronomicznie wiele. Zamiast tego **naśladujemy dobór naturalny**:

- 1 Trzymamy **zbiór gotowych rozwiązań** (każde to kompletny plan tras).
- 2 Bierzemy dwa dobre rozwiązania i „**krzyżujemy**” je — powstaje nowe, dziedziczące fragmenty po obojgu „rodzicach”.
- 3 Nowe rozwiązanie **podszlifowujemy** drobnymi poprawkami (przeszukiwanie lokalne).
- 4 Najlepsze przeżywają, słabe odrzucamy. **Powtarzamy tysiące razy.**

Jak wygląda jedna „tura” algorytmu



Trzy osie zrównoleglenia

Dodaliśmy do HGS trzy niezależne (i łączalne) mechanizmy:

1. GPU SWAP* (CUDA)

Ewaluacja par klientów dla SWAP* przeniesiona na GPU. Tysiące par liczonych równoległe, strumienie CUDA.

2. Master–slave (OpenMP)

Równoległe wytwarzanie potomstwa: 16 potomków na rundę, 8 wątków. „Many offspring”.

3. Model wysp (MPI)

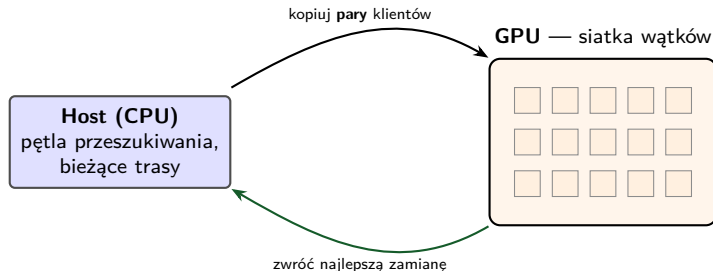
8 niezależnych populacji (wysp), topologia gwiazdy, polityki migracji i selekcji migrantów.

Dodatkowo: warstwa **telemetrii** i **monitor sprzętu** (zużycie CPU/GPU) do rzetelnej oceny.

Z tych mechanizmów złożyliśmy **6 wariantów testowych**: baseline, gpu, offspring16t8, gpu_offspring16t8, island8_star, island8_star_gpu_off16t8.

Oś 1: SWAP* na GPU — pomysł

Co chcemy przyspieszyć? W każdym „szlifie” rozwiązania algorytm sprawdza **ogromną liczbę par klientów** z różnych tras: czy ich zamiana skróci trasy? To miliony niezależnych, identycznych obliczeń — *idealne dla GPU*, które liczy tysiące takich par naraz.



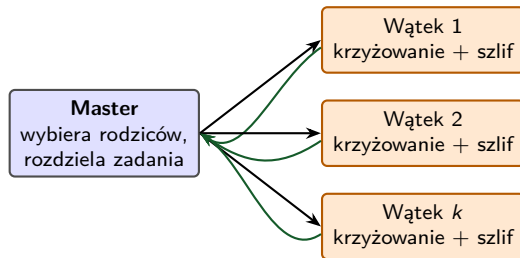
Każda komórka GPU ocenia jedną parę klientów; CPU dostaje gotowy ranking najlepszych zamian.

Co zbudowaliśmy (CUDA)

- **kernel** liczący koszt zamiany dla wszystkich par równolegle,
- **spłaszczony układ danych** tras (`GpuDataLayout`) dla szybkiego dostępu do pamięci GPU,
- **strumienie CUDA** — nakładanie obliczeń i transferów,
- pomiar zużycia GPU przez **NVML**.

Oś 2: model master-slave — pomysł

Co chcemy przyspieszyć? Najdroższy krok tury to „szlif” potomka (przeszukiwanie lokalne). Skoro mamy wiele rdzeni CPU — **wyprodukujemy wielu potomków naraz**, każdy szlifowany na osobnym wątku.



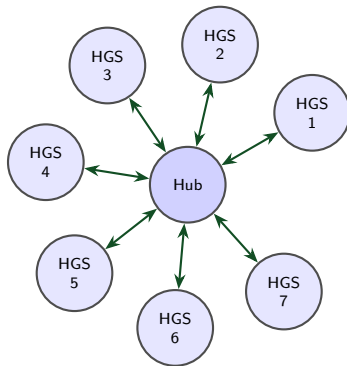
W naszym wariantcie: **16 potomków na rundę, 8 wątków.**

Co zbudowaliśmy (OpenMP)

- wspólny interfejs `OffspringMaker` z dwiema implementacjami: `Standard` (sekwencyjna) i `Parallel`,
- `ParallelOffspringMaker` — master rozdziela zadania, wątki liczą krzyżowanie + Split + szlif,
- sterowanie parametrami: liczba potomków i liczba wątków,
- dodatkowo: równoległa ścieżka SWAP* na CPU (`OpenMPLocalSearch`).

Oś 3: model wysp — pomysł

Inne podejście do równoległości: zamiast jednej populacji uruchamiamy **kilka niezależnych HGS** („wysp”) obok siebie. Co jakiś czas wyspy **wymieniają się najlepszymi rozwiązaniami** (migracja), by nawzajem się inspirować.



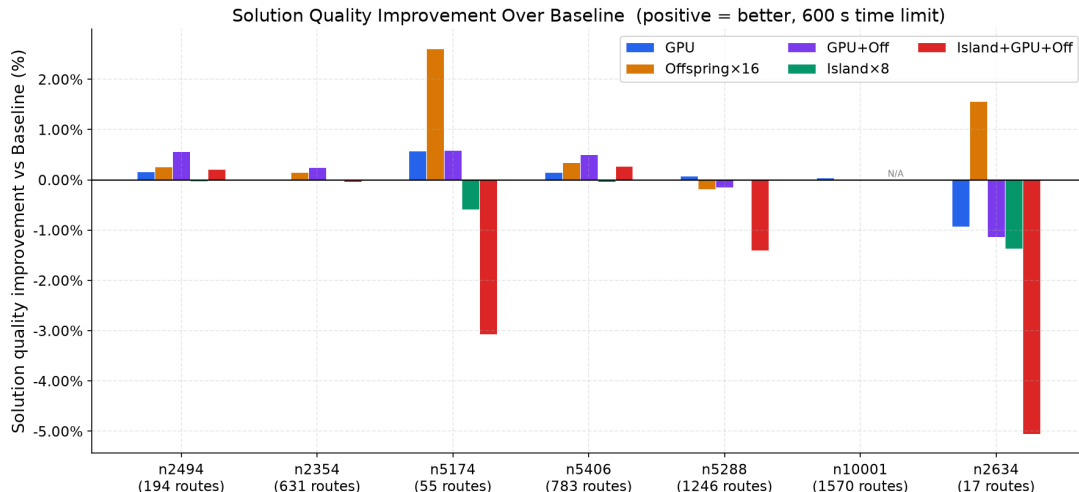
Topologia **gwiazdy**: 8 wysp wymienia migrantów przez centralny węzeł. Każda wyspa = **pełny HGS** na własnej podpopulacji.

Co zbudowaliśmy (MPI, modularnie)

- wymienne **topologie** połączeń (m.in. gwiazda),
- **polityki migracji** — kiedy i jak często wysyłać,
- **selektory migrantów** — kogo wysłać (np. najlepszego),
- obsługa imigrantów i **komunikator** (synchroniczny, MPI).

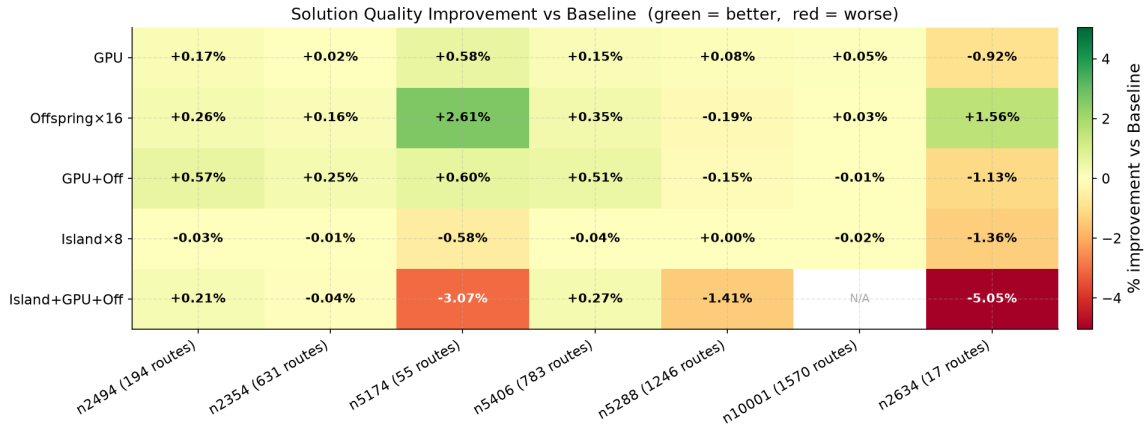
- **5 instancji XL** (2354–10001 klientów), **6 wariantów**, **limit czasu 600 s**, seed 1.
- Mierzone: jakość (koszt), liczba tur algorytmu, ewaluowane pary SWAP*, ruchy lokalne, wykorzystanie CPU/GPU, migracje, rundy potomstwa.

Poprawa jakości względem baseline



Wartość dodatnia = lepiej niż baseline. Skala efektu: ułamki procenta.

Mapa cieplna jakości — gdzie jest zielono, a gdzie czerwono



Najbardziej złożony wariant (wyspy+GPU+offspring) jest często **czerwony** — gorszy od baseline.

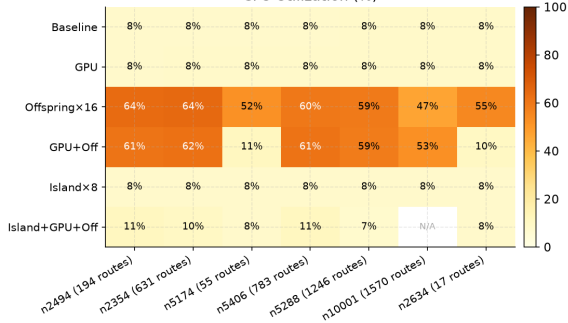
Liczby — najlepszy wariant na każdej instancji

Instancja	Baseline	Najlepszy wariant	Δ	Który wariant
XL-n2354-k631	947 790	945 389	+0,25%	GPU+Off
XL-n2494-k194	366 131	364 057	+0,57%	GPU+Off
XL-n2634-k17	33 220	32 701	+1,56%	Offspring
XL-n5174-k55	65 467	63 756	+2,61%	Offspring
XL-n5288-k1246	1 984 069	1 982 432	+0,08%	GPU
XL-n5406-k783	1 058 333	1 052 987	+0,51%	GPU+Off
XL-n10001-k1570	2 363 410	2 362 345	+0,05%	GPU

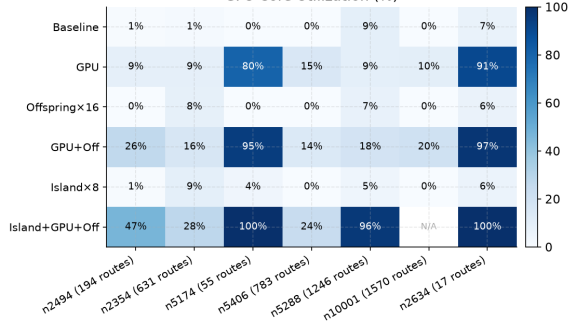
Wykorzystanie sprzętu — sedno problemu

Hardware Utilization per Preset and Instance

CPU Utilization (%)



GPU Core Utilization (%)



Dlaczego GPU ożywa przy małej liczbie pojazdów?

Karta jest wydajna tylko wtedy, gdy ma **dużo równoległej pracy na jedno wywołanie**.

- SWAP* porównuje **pary klientów** wewnątrz i między trasami — liczba par rośnie z **kwadratem długości trasy**.
- **Mało pojazdów** $\Rightarrow$ trasy bardzo **długie** (nawet ~ 155 klientów/trasę) $\Rightarrow$ ogromne, gęste bloki pracy $\Rightarrow$ **rdzenie GPU zajęte**.
- **Dużo pojazdów** $\Rightarrow$ trasy **malutkie** (3–4 klientów) $\Rightarrow$ pracy jak na lekarstwo $\Rightarrow$ dominuje narzut uruchomienia kernela i transferu $\Rightarrow$ **GPU głoduje**.

Instancja	klienci /trasę	rdzenie GPU
XL-n2634-k17	155	91%
XL-n5174-k55	94	80%
XL-n2494-k194	13	9%
XL-n5406-k783	7	15%
XL-n5288-k1246	4	9%
XL-n2354-k631	4	9%

„k” w nazwie = liczba pojazdów. Średnia długość trasy = n/k .

Czy to się opłaca? Bilans kosztów i korzyści

Korzyść

- poprawa jakości: $< 1\%$ (mediana), niestabilna,
- czasem **pogorszenie** jakości.

Koszt

- karta GPU (capex) + 8 rdzeni CPU + środowisko MPI,
- $3\times$ większa złożoność kodu (CUDA + OpenMP + MPI),
- utrzymanie, debugowanie, sprzęt.

Wniosek biznesowy Płacimy znaczną cenę sprzętową i inżynierską za zysk jakości na poziomie szumu. **Stosunek koszt/korzyść jest niekorzystny.**

Zużycie energii — perspektywa „perf-per-watt”

Szacunek poglądowy (rzędy wielkości, ten sam czas 600 s):

Wariant	Pobór mocy	Energia (600 s)	Zysk jakości
Baseline (1 rdzeń)	50 W	8 Wh	—
Offspring (8 rdzeni)	130 W	22 Wh	~+1%
GPU + offspring	280 W	47 Wh	~+0,5%
Wyspy + GPU + offspring	300 W	50 Wh	często < 0

Pobór mocy mierzony za pomocą urządzenia monitorującego pobór mocy z gniazdka. Wartości uśrednione dla różnych instancji.

Niestety nasze podejście do problemu było wadliwe od samego początku — próbowaliśmy **dopasować algorytm do równoległości**, zamiast **dopasować równoległość do algorytmu**. HGS jest zbudowany wokół sekwencyjnej, adaptacyjnej pętli, która nie daje się łatwo rozbić na niezależne kawałki. Próby zrównoleglenia tej pętli **nie poprawiły jakości** w sposób istotny, a czasem ją pogorszyły — bo wąskim gardłem nie jest przepustowość, lecz algorytm i struktura zależności danych.

Lepszym podejściem byłoby zbadanie algorytmów polegających na iteracyjnym ulepszaniu rozwiązania, które są **zaprojektowane z myślą o równoległości**

Czego się nauczyliśmy?

- Lepiej poznaliśmy mechanizmy działania algorytmów genetycznych i metaheurystycznych.
- Nauczyliśmy się technologii równoległych, szczególnie openMP oraz MPI, i ich zastosowania w praktyce.
- Chociaż nie udało się zawrzeć tego w końcowym raporcie, to mieliśmy możliwość skorzystania z dużego klastra obliczeniowego i nauczania się podstaw pracy z systemem kolejkowym, zarządzania zadaniami i monitorowania zasobów.

Dziękujemy za uwagę.